

PYTHON RECURSION — COMPLETE HANDWRITTEN-STYLE NOTES

Beginner → DSA Level

PART 1 — CORE CONCEPT

What is Recursion?

Recursion is a technique where a function calls itself to solve a smaller version of the same problem.

```
def countdown(n):  
    if n == 0: # base case  
        return  
    print(n)  
    countdown(n - 1) # recursive call  
  
countdown(3)
```

Output: 3 2 1

Every recursion needs two parts:

- **Base case** — tells the function when to stop.
- **Recursive case** — calls the same function with a smaller/simpler input.
- The recursive call must move toward the base case.

Call Stack & Unwinding

Each active function call is placed on the call stack. After the base case, calls return in reverse order. This is called unwinding.

```
factorial(4)  
→ 4 × factorial(3)  
→ 4 × 3 × factorial(2)  
→ 4 × 3 × 2 × factorial(1)  
→ 24
```

Recursion vs Loop

Recursion	Loop
Function calls itself	for/while repeats
Uses call stack	Usually less stack usage
Natural for trees/backtracking	Often simpler for repetition
Can be elegant but depth-limited	Usually predictable memory

PART 2 — CLASSIC RECURSION PROBLEMS

1. Factorial

$n! = n \times (n-1) \times \dots \times 1$, and $0! = 1$.

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5)) # 120
```

2. Sum from 1 to n

```
def sum_n(n):
    if n == 0:
        return 0
    return n + sum_n(n - 1)

print(sum_n(5)) # 15
```

3. Power

```
def power(a, n):
    if n == 0:
        return 1
    return a * power(a, n - 1)

print(power(2, 5)) # 32
```

4. Reverse a String

```
def reverse(s):
    if len(s) <= 1:
        return s
    return reverse(s[1:]) + s[0]

print(reverse("python")) # nohtyp
```

5. Sum of a List

```
def list_sum(nums):
    if not nums:
        return 0
    return nums[0] + list_sum(nums[1:])

print(list_sum([1, 2, 3, 4])) # 10
```

Core pattern: Base case → process current value → solve the smaller problem.

PART 3 — FIBONACCI & OPTIMIZATION

Fibonacci

Sequence: 0, 1, 1, 2, 3, 5, 8, ...

$F(n) = F(n-1) + F(n-2)$.

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(6)) # 8
```

Naive Fibonacci repeats the same subproblems many times, giving approximately $O(2^n)$ time.

Memoization

```
def fib(n, memo=None):
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]

print(fib(40))
```

Memoization stores already-computed results. Time becomes approximately $O(n)$ with $O(n)$ extra space.

Bottom-Up Alternative

```
def fib_iter(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Important: recursion is not automatically better. Choose the clearest approach with acceptable complexity.

Mutable default warning: avoid `memo={}` as a default. Use `memo=None` and create the dictionary inside.

PART 4 — CALL STACK, SCOPE & COMPLEXITY

Call Stack Example

```
def show(n):  
    if n == 0:  
        return  
    print("down", n)  
    show(n - 1)  
    print("up", n)  
  
show(3)
```

The function first goes down through recursive calls, then comes back up while returning.

Recursion Limit

```
import sys  
print(sys.getrecursionlimit())
```

Python has a recursion depth limit. Very deep recursion can raise **RecursionError**. Increasing the limit is not a substitute for a safe algorithm.

Typical Complexity

Pattern	Time	Extra Stack
One recursive call/level	$O(n)$	$O(n)$
Naïve Fibonacci	$O(2^n)$ approx.	$O(n)$ depth
Memoized Fibonacci	$O(n)$	$O(n)$

When recursion is useful: trees, divide-and-conquer, backtracking, subsets, permutations, and naturally recursive problems.

PART 5 — RECURSION FOR DSA & BACKTRACKING

Backtracking Pattern

The key idea is: **Choose** → **Recurse** → **Undo**.

```
def generate(current, choices):
    if len(current) == len(choices):
        print(current)
        return

    for x in choices:
        if x not in current:
            current.append(x) # choose
            generate(current, choices)
            current.pop() # undo
```

Generate Subsets

```
def subsets(nums, i=0, path=None):
    if path is None:
        path = []
    if i == len(nums):
        print(path)
        return

    path.append(nums[i])
    subsets(nums, i + 1, path)

    path.pop()
    subsets(nums, i + 1, path)
```

For n elements there are 2^n possible subsets.

Generate Permutations

```
def permute(nums, start=0):
    if start == len(nums):
        print(nums)
        return

    for i in range(start, len(nums)):
        nums[start], nums[i] = nums[i], nums[start]
        permute(nums, start + 1)
        nums[start], nums[i] = nums[i], nums[start] # undo
```

Tree Traversal

```
def preorder(node):
    if node is None:
        return
    print(node.val)
    preorder(node.left)
    preorder(node.right)
```

DSA connection: DFS, tree traversal, divide-and-conquer, subsets, permutations, combinations and backtracking all rely heavily on recursion.

PART 6 — MISTAKES, DEBUGGING & PRACTICE

Common Mistakes

- **No base case** — recursion never stops.
- **Unreachable base case** — valid input never reaches it.
- **Same input again** — $f(n)$ calls $f(n)$, causing infinite recursion.
- **Wrong return** — forgetting to return the recursive result can produce `None`.
- **Repeated work** — use memoization when subproblems repeat.
- **Too deep** — recursion has a depth limit.
- **Unnecessary recursion** — sometimes a loop is simpler.

Debugging Recursion

```
def factorial(n):  
    print("enter:", n)  
    if n <= 1:  
        return 1  
  
    result = n * factorial(n - 1)  
    print("return:", n, result)  
    return result
```

Print the input when entering and the result before returning. This makes the call stack easier to understand.

Beginner Practice

- Print n to 1 and 1 to n .
- Find factorial.
- Find sum from 1 to n .
- Calculate a^n .
- Reverse a string.
- Find the sum of list elements.
- Check whether a string is a palindrome.

Intermediate / DSA Practice

- Find maximum element recursively.
- Count occurrences in a list.
- Implement recursive binary search.
- Implement recursive GCD.
- Generate all subsets.
- Generate all permutations.
- Generate combinations.
- Traverse a binary tree.
- Solve a maze/path problem.
- Try N-Queens using backtracking.

QUICK REVISION

RECURSION = FUNCTION CALLS ITSELF

Every recursion problem:

1. BASE CASE → STOP
2. RECURSIVE CASE → SMALLER PROBLEM
3. RETURN → COMBINE RESULT

BACKTRACKING:

Choose → Recurse → Undo

Final Checklist

- What is my base case?
- Does every call move toward it?
- How deep can the call stack become?
- What is the time complexity?
- What is the extra space?
- Would a loop be simpler?
- Are there repeated subproblems?

■ NEVER GIVE UP JAHANUR, YOU CAN DO IT!